UNIT 3

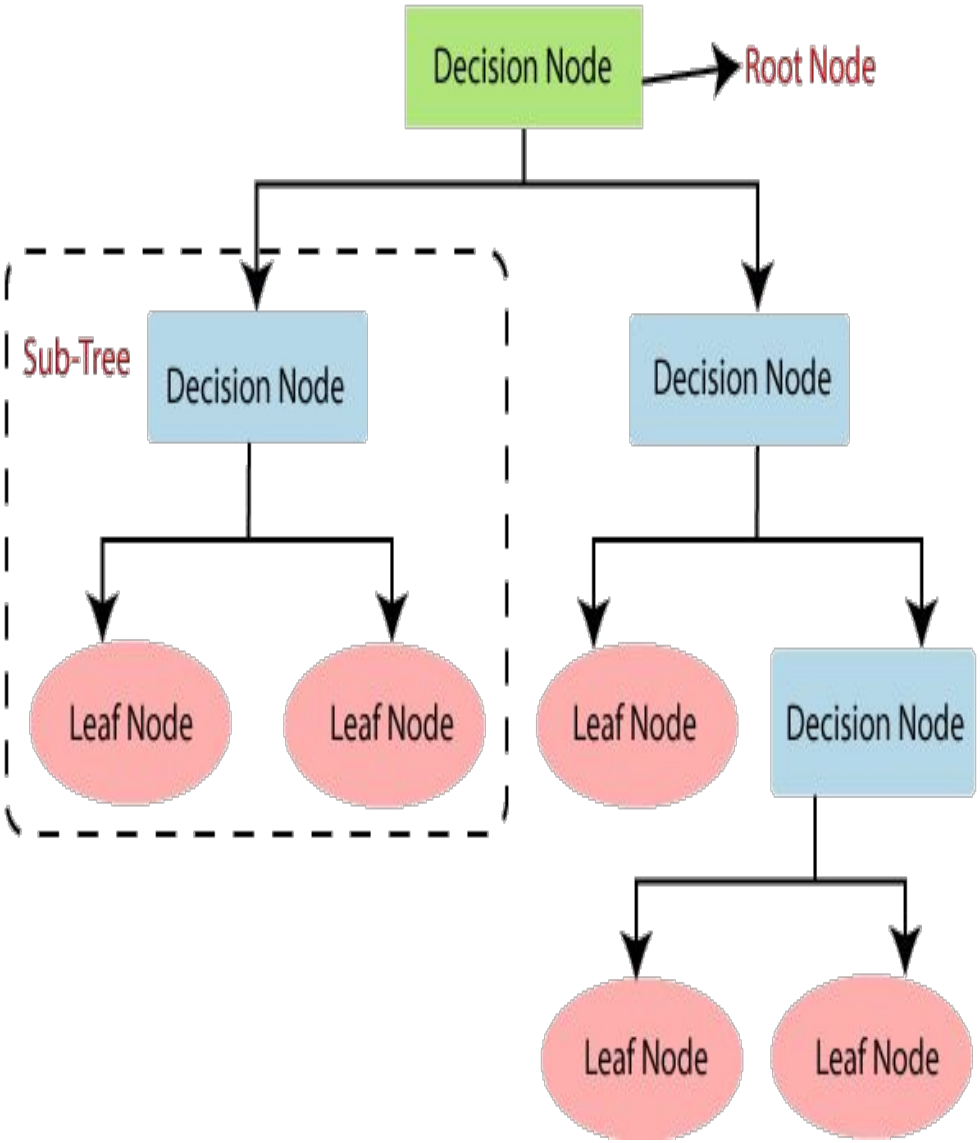
Learning with Trees

- We are now going to consider a rather different approach to machine learning, starting with one of the most common and powerful data structures in the whole of computer science:
- The binary tree :the computational cost of making the tree is fairly low, but the cost of using it is even lower: O(log*N), where N is the number of datapoints.*
- *This is important for* machine learning, since querying the trained algorithm should be as fast as possible since it happens more often, and the result is often wanted immediately.
- This is sufficient to make trees seem attractive for machine learning and they are easy to understand.

Decision Trees

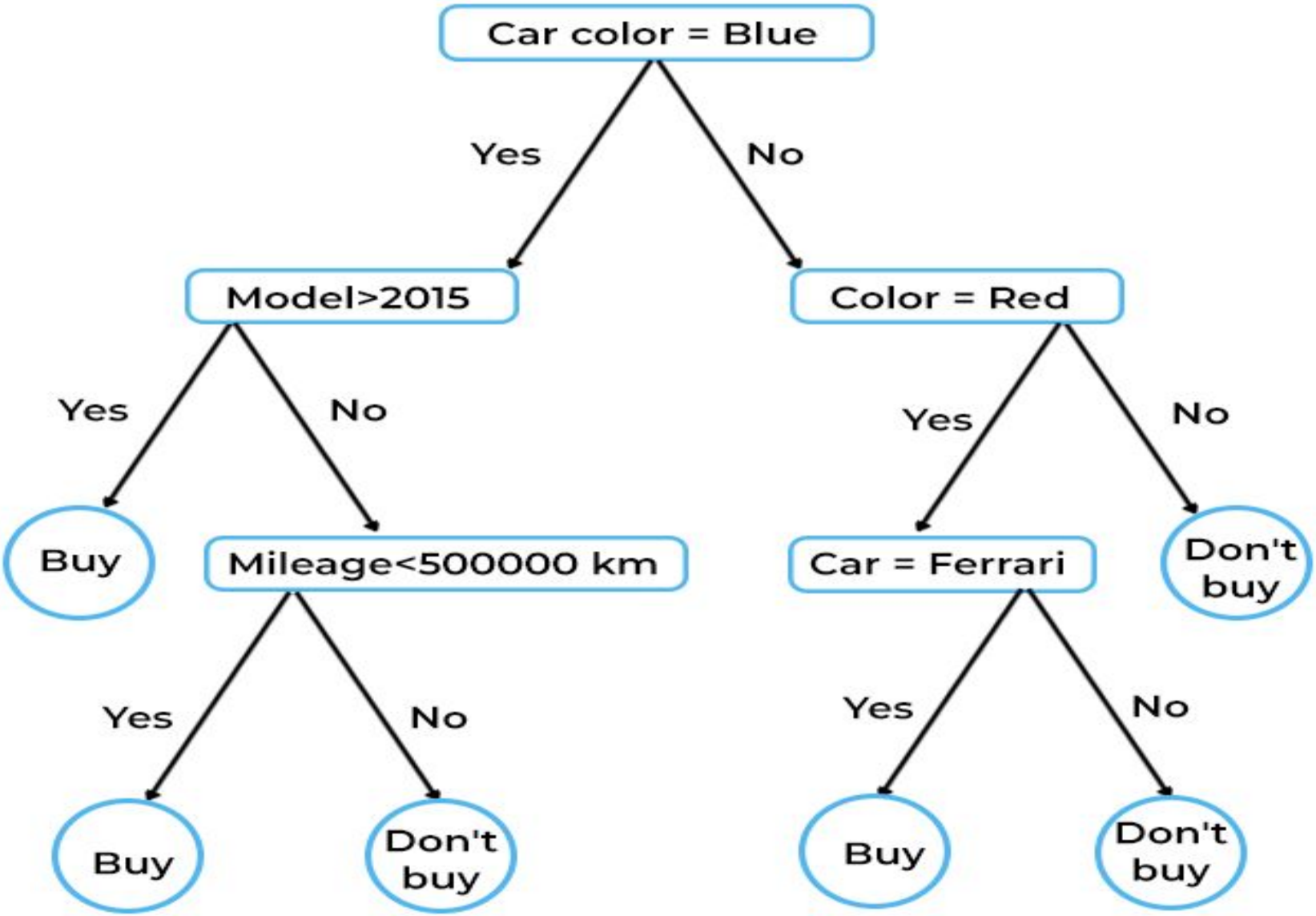
- Decision Tree is a **Supervised learning technique** that can be used for both classification and Regression problems, but mostly it is preferred for solving Classification problems.
- It is a tree-structured classifier, where
  - **internal nodes represent the features of a dataset,**
  - **branches represent the decision rules** and
  - **each leaf node represents the outcome.**
- In a Decision tree, there are two nodes, which are the **Decision Node** and **Leaf Node.**

- Decision nodes are used to make any decision and have multiple branches, whereas Leaf nodes are the output of those decisions and do not contain any further branches.
- The decisions or the test are performed on the basis of features of the given dataset.



TOOLBOX™

BUYING A CAR



- The idea of a decision tree is that we break classification down into a set of choices about each feature in turn, starting at the root (base) of the tree and progressing down to the leaves, where we receive the classification decision.

- The trees are very easy to understand, and can even be turned into a set of if-then rules, suitable for use in a rule induction system.

- In terms of optimisation and search, decision trees use a **greedy heuristic** to perform search, evaluating the possible options at the current stage of learning and making the one that seems optimal at that point.
- This works well a surprisingly large amount of the time.

Using Decision Tree

- As a student it can be difficult to decide what to do in the evening.
- There are four things that you actually quite enjoy doing, or have to do:
  - going to the pub,
  - watching TV,
  - going to a party, or
  - even (gasp) studying.
- The choice is sometimes made for you—if you have an assignment due the next day, then you need to study, if you are feeling lazy then the pub isn't for you, and if there isn't a party then you can't go to it. You are looking for a nice algorithm that will let you decide what to do each evening without having to think about it every night.

- Each evening you start at the top (root) of the tree and check whether any of your friends know about a party that night. If there is one, then you need to go, regardless.

- Only if there is not a party do you worry about whether or not you have an assignment deadline coming up. If there is a crucial deadline, then you have to study, but if there is nothing that is urgent for the next few days, you think about how you feel.

- A sudden burst of energy might make you study, but otherwise you'll be slumped in front of the TV indulging your favorite movie or series rather than studying.

- Of course, near the start of the semester when there are no assignments to do, and you are feeling rich, you'll be in the pub.

- One of the reasons that decision trees are popular is that we can turn them into a set of logical disjunctions (if ... then rules) that then go into program code very simply—the first part of the tree above can be turned into:

- if *there is a party then go to it*
- if *there is not a party and you have an urgent deadline then study*
- etc.

CONSTRUCTING DECISION TREES

Classification by Decision Tree Induction

- Decision tree
  - A flow-chart-like tree structure
  - Internal node denotes a test on an attribute
  - Branch represents an outcome of the test
  - Leaf nodes represent class labels or class distribution

- Decision tree generation consists of two phases
  - Tree construction
    - At start, all the training examples are at the root
    - Partition examples recursively based on selected attributes
  - Tree pruning
    - Identify and remove branches (nodes) that reflect noise or outliers

- Use of decision tree: Classifying an unknown sample
  - Test the attribute values of the sample against the decision tree

ID3 (Iterative Dichotomiser 3)

The ID3 Algorithm

- If all examples have the same label:
  - return a leaf with that label
- Else if there are no features left to test:
  - return a leaf with the most common label
- Else:
  - choose the feature $\hat{F}$ that maximises the information gain of S to be the next node using Equation (12.2)
  - add a branch from the node for each possible value f in $\hat{F}$
  - for each branch:
    - calculate S_f by removing $\hat{F}$ from the set of features
    - recursively call the algorithm with S_f, to compute the gain relative to the current set of examples

- When we want to construct the decision tree to decide what to do in the evening

Deadline?	Is there a party?	Lazy?	Activity
Urgent	Yes	Yes	Party
Urgent	No	Yes	Study
Near	Yes	Yes	Party
None	Yes	No	Party
None	No	Yes	Pub
None	Yes	No	Party
Near	No	No	Study
Near	No	Yes	TV
Near	Yes	Yes	Party
Urgent	No	No	Study

12.2.1 Quick Aside: Entropy in Information Theory

Information theory was 'born' in 1948 when Claude Shannon published a paper called "A Mathematical Theory of Communication." In that paper, he proposed the measure of information entropy, which describes the amount of impurity in a set of features. The entropy H of a set of probabilities p_i is (for those who know some physics, the relation to physical entropy should be clear):

$$\text{Entropy}(p) = -\sum_i p_i \log_2 p_i, \tag{12.1}$$

where the logarithm is base 2 because we are imagining that we encode everything using binary digits (bits), and we define $0 \log 0 = 0$. A graph of the entropy is given in Figure 12.2. Suppose that we have a set of positive and negative examples of some feature (where the feature can only take 2 values: positive and negative). If all of the examples are positive, then we don't get any extra information from knowing the value of the feature for any particular example, since whatever the value of the feature, the example will be positive. Thus, the entropy of that feature is 0. However, if the feature separates the examples into 50% positive and 50% negative, then the amount of entropy is at a maximum, and knowing about that feature is very useful to us. The basic concept is that it tells us how much *extra* information we would get from knowing the value of that feature. A function for computing

- In the Decision Tree, the major challenge is the identification of the attribute for the root node at each level. This process is known as attribute selection measure (ASM). We have two popular attribute selection measures:

1. Information Gain
2. Gini Index

1. Information Gain:

When we use a node in a decision tree to partition the training instances into smaller subsets the entropy changes. Information gain is a measure of this change in entropy.

Gini index:

Gini Index is a metric to measure how often a randomly chosen element would be incorrectly identified.

- It means an attribute with a lower Gini index should be preferred.
- Sklearn supports “Gini” criteria for Gini Index and by default, it takes “gini” value.
- The Formula for the calculation of the Gini Index is given below.

the entropy of the whole set minus the entropy when a particular feature is chosen. This is defined by (where S is the set of examples, F is a possible feature out of the set of all possible ones, and $|S_f|$ is a count of the number of members of S that have value f for feature F):

$$\text{Gain}(S, F) = \text{Entropy}(S) - \sum_{f \in values(F)} \frac{|S_f|}{|S|} \text{Entropy}(S_f). \tag{12.2}$$

As an example, suppose that we have data (with outcomes) $S = \{s_1 = \text{true}, s_2 = \text{false}, s_3 = \text{false}, s_4 = \text{false}\}$ and one feature F that can have values $\{f_1, f_2, f_3\}$. In the example, the feature value for s_1 could be f_2, for s_2 it could be f_2, for s_3, f_3 and for s_4, f_1 then we can calculate the entropy of S as (where $\oplus$ means true, of which we have one example, and $\ominus$ means false, of which we have three examples):

$$\begin{aligned} \text{Entropy}(S) &= -p_\oplus \log_2 p_\oplus - p_\ominus \log_2 p_\ominus \\ &= -\frac{1}{4} \log_2 \frac{1}{4} - \frac{3}{4} \log_2 \frac{3}{4} \\ &= 0.5 + 0.311 = 0.811. \end{aligned} \tag{12.3}$$

The function $\text{Entropy}(S_f)$ is similar, but only computed with the subset of data where feature F has values f.

$$
\begin{aligned}
\frac{|S_{f_1}|}{|S|}\text{Entropy}(S_{f_1}) &= \frac{1}{4} \times \left(-\frac{0}{1}\log_2\frac{0}{1} - \frac{1}{1}\log_2\frac{1}{1} \right) \\
&= 0 \\
\frac{|S_{f_2}|}{|S|}\text{Entropy}(S_{f_2}) &= \frac{2}{4} \times \left(-\frac{1}{2}\log_2\frac{1}{2} - \frac{1}{2}\log_2\frac{1}{2} \right) \\
&= \frac{1}{2} \\
\frac{|S_{f_3}|}{|S|}\text{Entropy}(S_{f_3}) &= \frac{1}{4} \times \left(-\frac{0}{1}\log_2\frac{0}{1} - \frac{1}{1}\log_2\frac{1}{1} \right) \\
&= 0
\end{aligned}
$$

The information gain from adding this feature is the entropy of S minus the sur three values above:

$$\text{Gain}(S, F) = 0.811 - (0 + 0.5 + 0) = 0.311.$$

To produce a decision tree for this problem, the first thing that we need to do is work out which feature to use as the root node. We start by computing the entropy of S:

$$\begin{aligned}
\text{Entropy}(S) &= -p_{\text{party}} \log_2 p_{\text{party}} - p_{\text{study}} \log_2 p_{\text{study}} \\
&- p_{\text{pub}} \log_2 p_{\text{pub}} - p_{\text{TV}} \log_2 p_{\text{TV}} \\
&= -\frac{5}{10} \log_2 \frac{5}{10} - \frac{3}{10} \log_2 \frac{3}{10} - \frac{1}{10} \log_2 \frac{1}{10} - \frac{1}{10} \log_2 \frac{1}{10} \\
&= 0.5 + 0.5211 + 0.3322 + 0.3322 = 1.6855 \qquad (12.11)
\end{aligned}$$

and then find which feature has the maximal information gain:

$$
\begin{aligned}
\text{Gain}(S, \text{Deadline}) &= 1.6855 - \frac{|S_{\text{urgent}}|}{10}\text{Entropy}(S_{\text{urgent}}) \\
&- \frac{|S_{\text{near}}|}{10}\text{Entropy}(S_{\text{near}}) - \frac{|S_{\text{none}}|}{10}\text{Entropy}(S_{\text{none}}) \\
&= 1.6855 - \frac{3}{10}\left(-\frac{2}{3}\log_2\frac{2}{3} - \frac{1}{3}\log_2\frac{1}{3}\right) \\
&- \frac{4}{10}\left(-\frac{2}{4}\log_2\frac{2}{4} - \frac{1}{4}\log_2\frac{1}{4} - \frac{1}{4}\log_2\frac{1}{4}\right) \\
&- \frac{3}{10}\left(-\frac{1}{3}\log_2\frac{1}{3} - \frac{2}{3}\log_2\frac{2}{3}\right) \\
&= 1.6855 - 0.2755 - 0.6 - 0.2755 \\
&= 0.5345
\end{aligned}
$$

$$
\begin{aligned}
\text{Gain}(S, \text{Party}) &= 1.6855 - \frac{5}{10}\left(-\frac{5}{5}\log_2\frac{5}{5}\right) \\
&- \frac{5}{10}\left(-\frac{3}{5}\log_2\frac{3}{5} - \frac{1}{5}\log_2\frac{1}{5} - \frac{1}{5}\log_2\frac{1}{5}\right) \\
&= 1.6855 - 0 - 0.6855 \\
&= 1.0
\end{aligned}
$$

$$
\begin{aligned}
\text{Gain}(S, \text{Lazy}) &= 1.6855 - \frac{6}{10}\left(-\frac{3}{6}\log_2\frac{3}{6} - \frac{1}{6}\log_2\frac{1}{6} - \frac{1}{6}\log_2\frac{1}{6} - \frac{1}{6}\log_2\frac{1}{6}\right) \\
&- \frac{4}{10}\left(-\frac{2}{4}\log_2\frac{2}{4} - \frac{2}{4}\log_2\frac{2}{4}\right) \\
&= 1.6855 - 1.0755 - 0.4 \\
&= 0.21
\end{aligned}
$$

Therefore, the root node will be the party feature, which has two feature values (‘yes’ and ‘no’), so it will have two branches coming out of it (see Figure 12.6).

When we look at the ‘yes’ branch, we see that in all five cases where there was a party we went to it, so we just put a leaf node there, saying ‘party’.

For the ‘no’ branch, out of the five cases there are three different outcomes, so now we need to choose another feature. The five cases we are looking at are:

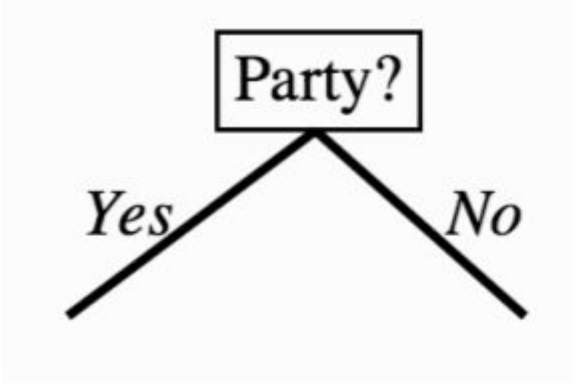


FIGURE 12.6 The decision tree after one step of the algorithm.

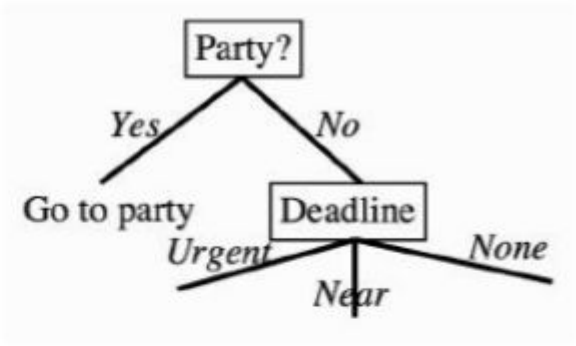


FIGURE 12.7 The tree after another step.

Deadline?	Is there a party?	Lazy?	Activity
Urgent	No	Yes	Study
None	No	Yes	Pub
Near	No	No	Study
Near	No	Yes	TV
Urgent	No	Yes	Study

We've used the party feature, so we just need to calculate the information gain of the other two over these five examples:

$$
\begin{aligned}
\text{Gain}(S, \text{Deadline}) &= 1.371 - \frac{2}{5}\left(-\frac{2}{2}\log_2\frac{2}{2}\right) \\
&- \frac{2}{5}\left(-\frac{1}{2}\log_2\frac{1}{2} - \frac{1}{2}\log_2\frac{1}{2}\right) - \frac{1}{5}\left(-\frac{1}{1}\log_2\frac{1}{1}\right) \\
&= 1.371 - 0 - 0.4 - 0 \\
&= 0.971 \\
\text{Gain}(S, \text{Lazy}) &= 1.371 - \frac{4}{5}\left(-\frac{2}{4}\log_2\frac{2}{4} - \frac{1}{4}\log_2\frac{1}{4} - \frac{1}{4}\log_2\frac{1}{4}\right) \\
&- \frac{1}{5}\left(-\frac{1}{1}\log_2\frac{1}{1}\right) \\
&= 1.371 - 1.2 - 0 \\
&= 0.1710
\end{aligned}
$$

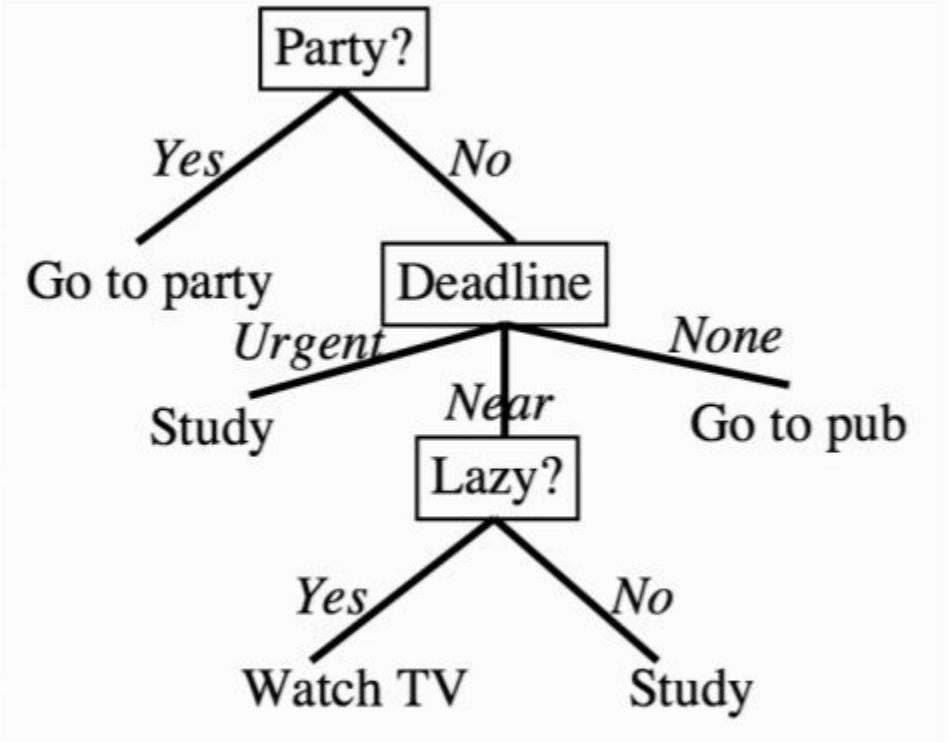


FIGURE 12.1 A simple decision tree to decide how you will spend the evening.

CART (Classification And Regression Tree)

- CART is a variation of the decision tree algorithm. It can handle both classification and regression tasks.
- Scikit-Learn uses the Classification And Regression Tree (CART) algorithm to train Decision Trees (also called “growing” trees).
- CART was first produced by Leo Breiman, Jerome Friedman, Richard Olshen, and Charles Stone in 1984.
- The entropy that was used in ID3 as the information measure is not the only way to pick features. Another possibility is something known as the Gini impurity.

CART (Classification And Regression Tree)

Gini Index/Gini Impurity:

The Gini index is a metric for the classification tasks in CART. It stores the sum of squared probabilities of each class. It computes the degree of probability of a specific variable that is wrongly being classified when chosen randomly and a variation of the Gini coefficient. It works on categorical variables, provides outcomes either “successful” or “failure” and hence conducts binary splitting only.

The degree of the Gini index varies from 0 to 1,

- Where 0 depicts that all the elements are allied to a certain class, or only one class exists there.
- The Gini index of value 1 signifies that all the elements are randomly distributed across various classes, and
- A value of 0.5 denotes the elements are uniformly distributed into some classes.

CART (Classification And Regression Tree)

Gini Index/Gini Impurity:

$$G_k = \sum_{i=1}^{c} \sum_{j \neq i} N(i)N(j),$$

where c is the number of classes. In fact, you can reduce the algorithmic effort required by noticing that P i N(i) = 1 (since there has to be some output class) and so P j6=i N(j) = 1 − N(i). Then Equation above is equivalent to:

$$G_k = 1 - \sum_{i=1}^{c} N(i)^2.$$

The term CART serves as a generic term for the following categories of decision trees:

- **Classification Trees:** The tree is used to determine which "class" the target variable is most likely to fall into when it is continuous.
- **Regression trees:** These are used to predict a continuous variable's value.

CART (Classification And Regression Tree)

CART for Regression

A Regression tree is an algorithm where the target variable is continuous and the tree is used to predict its value. Regression trees are used when the response variable is continuous. For example, if the response variable is the temperature of the day.

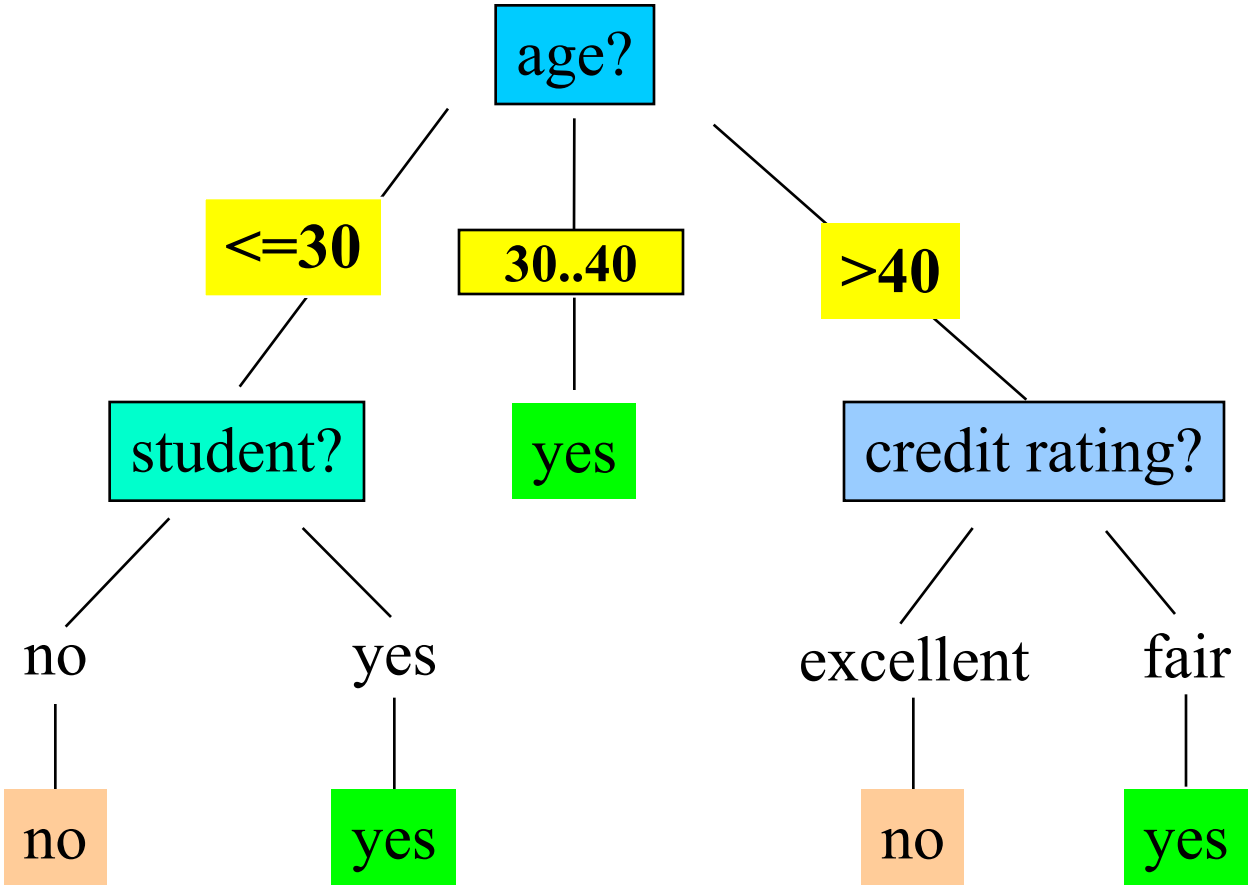
CART for regression is a decision tree learning method that creates a tree-like structure to predict continuous target variables. The tree consists of nodes that represent different decision points and branches that represent the possible outcomes of those decisions. Predicted values for the target variable are stored in each leaf node of the tree.

Training Dataset

This follows an example from Quinlan's ID3

age	income	student	credit_rating
<=30	high	no	fair
<=30	high	no	excellent
31…40	high	no	fair
>40	medium	no	fair
>40	low	yes	fair
>40	low	yes	excellent
31…40	low	yes	excellent
<=30	medium	no	fair
<=30	low	yes	fair
>40	medium	yes	fair
<=30	medium	yes	excellent
31…40	medium	no	excellent
31…40	high	yes	fair
>40	medium	no	excellent

Output: A Decision Tree for "*buys_computer*"



Algorithm for Decision Tree Induction

- Basic algorithm (a greedy algorithm)
  - Tree is constructed in a top-down recursive divide-and-conquer manner
  - At start, all the training examples are at the root
  - Attributes are categorical (if continuous-valued, they are discretized in advance)
  - Examples are partitioned recursively based on selected attributes
  - Test attributes are selected on the basis of a heuristic or statistical measure (e.g., information gain)
- Conditions for stopping partitioning
  - All samples for a given node belong to the same class
  - There are no remaining attributes for further partitioning – majority voting is employed for classifying the leaf
  - There are no samples left

Attribute Selection Measure

- Information gain (ID3/C4.5)
  - All attributes are assumed to be categorical
  - Can be modified for continuous-valued attributes
- Gini index (IBM IntelligentMiner)
  - All attributes are assumed continuous-valued
  - Assume there exist several possible split values for each attribute
  - May need other tools, such as clustering, to get the possible split values
  - Can be modified for categorical attributes

RID	Age	Income	Student	Credit_rating	Class : Buys_computer
1	=30	High	No	Fair	No
2	<=30	High	No	Excellent	No
3	31.40	High	No	Fair	Yes
4	>40	Medium	No	Fair	Yes
5	>40	Low	Yes	Fair	Yes
6	>40	Low	Yes	Excellent	No
7	31.40	Low	Yes	Excellent	Yes
8	<=30	Medium	No	Fair	No
9	<=30	Low	Yes	Fair	Yes
10	>40	Medium	Yes	Fair	Yes
11	<=30	Medium	Yes	Excellent	Yes
12	31.40	Medium	No	Excellent	Yes
13	31.40	High	Yes	Fair	Yes
14	>40	Medium	No	Excellent	No

Information Gain (ID3/C4.5)

- Select the attribute with the highest information gain
- Assume there are two classes, *P* and *N*
  - Let the set of examples *S* contain *p* elements of class *P* and *n* elements of class *N*
  - The amount of information, needed to decide if an arbitrary example in *S* belongs to *P* or *N* is defined as

$$I(p,n) = -\frac{p}{p+n}\log_2\frac{p}{p+n} - \frac{n}{p+n}\log_2\frac{n}{p+n}$$

Information Gain in Decision Tree Induction

- Assume that using attribute A a set *S* will be partitioned into sets $\{S_1, S_2, \ldots, S_v\}$
  - If S_i contains p_i examples of *P* and n_i examples of *N*, the entropy, or the expected information needed to classify objects in all subtrees S_i is

$$E(A) = \sum_{i=1}^{v} \frac{p_i + n_i}{p + n} I(p_i, n_i)$$

- The encoding information that would be gained by branching on *A*

$$Gain(A) = I(p, n) - E(A)$$

Attribute Selection by Information Gain Computation

- Class P: buys_computer = "yes"
- Class N: buys_computer = "no"
- I(p, n) = I(9, 5) =0.940
- Compute the entropy for *age*:

age	p_i	n_i	$I(p_i, n_i)$
<=30	2	3	0.971
30…40	4	0	0
>40	3	2	0.971

$$E(age) = \frac{5}{14} I(2,3) + \frac{4}{14} I(4,0) + \frac{5}{14} I(3,2) = 0.69$$

Hence

$$Gain(age) = I(p,n) - E(age)$$

Similarly

$$Gain(income) = 0.029$$

$$Gain(student) = 0.151$$

$$Gain(credit\_rating) = 0.048$$

To compute the information gain of each attribute

I (S1,S2) = I(9,5)

$$I(p,n) = -\frac{p}{p+n}\log_2\frac{p}{p+n} - \frac{n}{p+n}\log_2\frac{n}{p+n}$$

= -9/14 log 9/14 – 5/14 log 5/14

$$= -9/14 \quad \frac{\log_{10}(9/14)}{\log_2(2)} - 5/14 \quad \frac{\log_{10}(5/14)}{\log_2(2)} -$$

= .6428 * .6374 + .3571 * 1.4854

= 0.4097 + 0.5304

= 0.9401

Extracting Classification Rules from Trees

- Represent the knowledge in the form of IF-THEN rules
- One rule is created for each path from the root to a leaf
- Each attribute-value pair along a path forms a conjunction
- The leaf node holds the class prediction
- Rules are easier for humans to understand
- Example

IF *age* = "<=30" AND *student* = "*no*" THEN *buys_computer* = "*no*"

IF *age* = "<=30" AND *student* = "*yes*" THEN *buys_computer* = "*yes*"

IF *age* = "31…40" THEN *buys_computer* = "*yes*"

IF *age* = ">40" AND *credit_rating* = "*excellent*" THEN *buys_computer* = "*yes*"

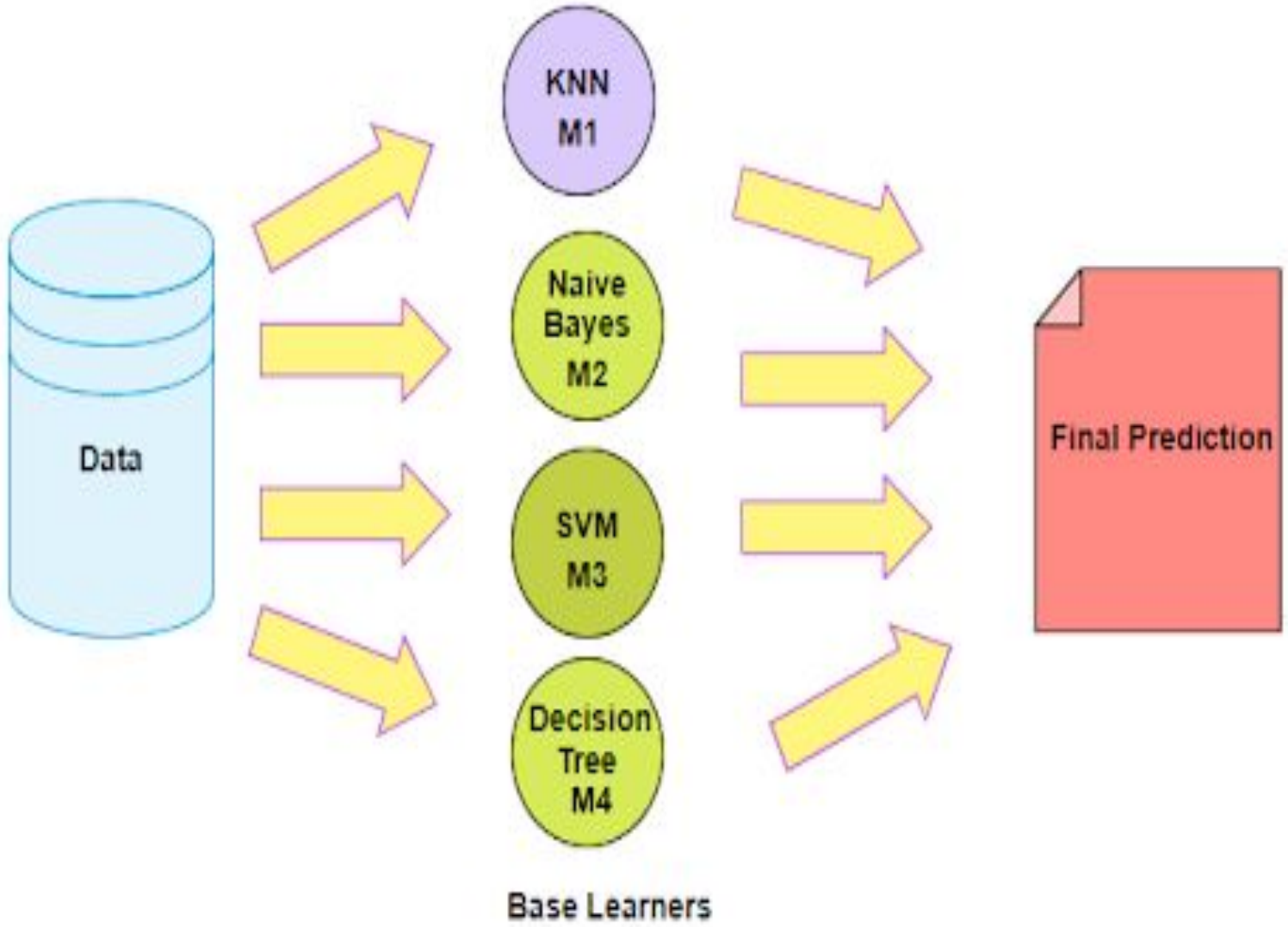
IF *age* = ">40" AND *credit_rating* = "*fair*" THEN *buys_computer* = "*no*"

- It is worth considering how ID3 generalises from training examples to the set of all possible inputs. It uses a method known as the inductive bias.
- The choice of the next feature to add into the tree is the one with the highest information gain, which biases the algorithm towards smaller trees, since it tries to minimise the amount of information that is left.
- This is consistent with a well-known principle that short solutions are usually better than longer ones.
- Note that the algorithm can deal with noise in the dataset, because the labels are assigned to the most common value of the target attribute.
- Another benefit of decision trees is that they can deal with missing data.

- Think what would happen if an example has a missing feature.
- In that case, we can skip that node of the tree and carry on without it, summing over all the possible values that that feature could have taken.
- This is virtually impossible to do with neural networks: how do you represent missing data when the computation is based on whether or not a neuron is firing?
- In the case of neural networks it is common to either throw away any datapoints that have missing data, or guess (more technically impute any missing values, either by identifying similar datapoints and using their value or by using the mean or median of the data values for that feature).
- This assumes that the data that is missing is randomly distributed within the dataset, not missing because of some unknown process.

Ensembling Learning

- Ensemble learning is a supervised learning algorithm
- To improve overall performance by combining the predictions from multiple models
- Meta algorithm which combine multiple ML techniques into one prediction model

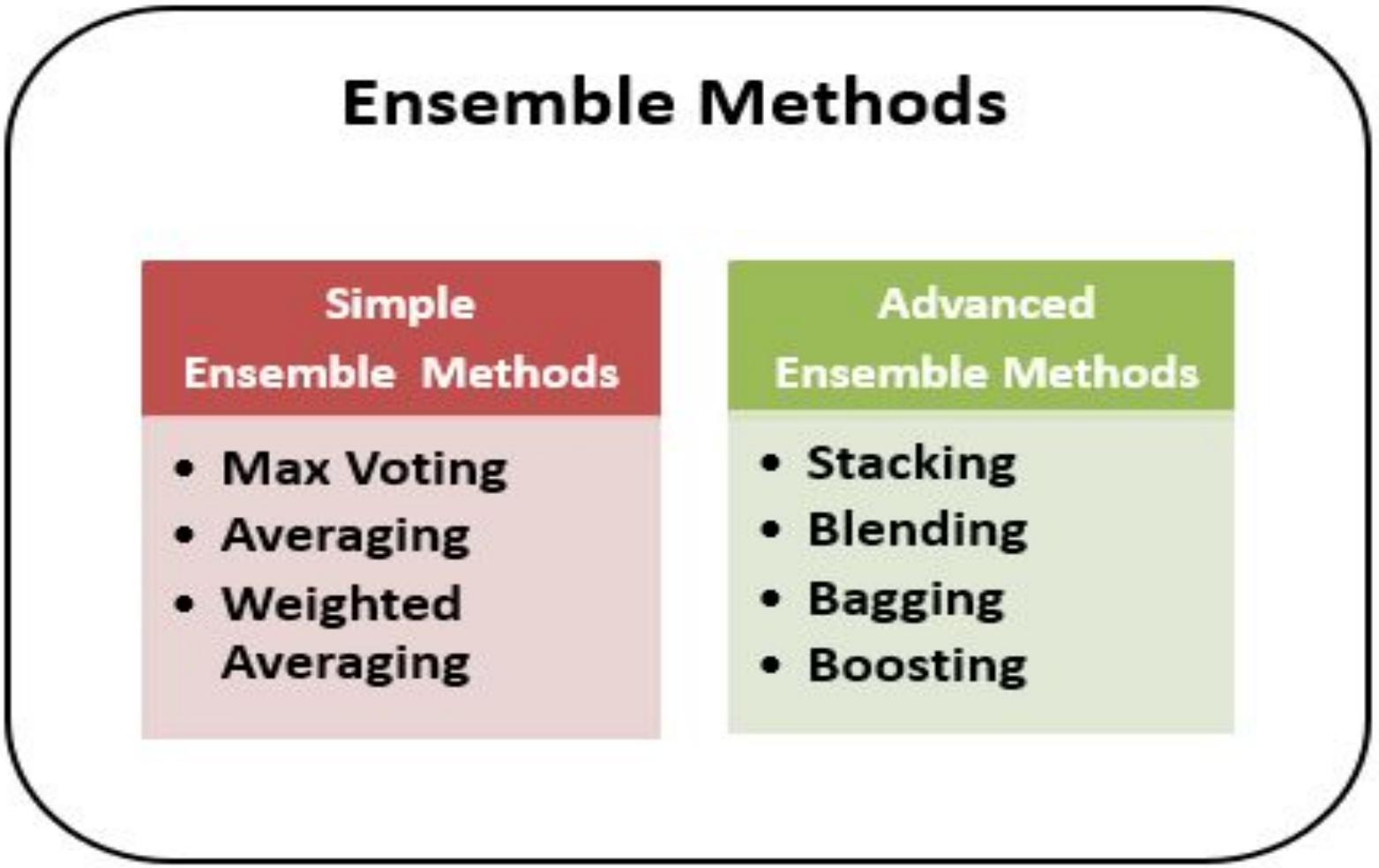

KNN
M1
Naive
Bayes
M2
SVM
M3
Decision
Tree
M4
Data
Final Prediction
Base Learners

Ensembling Learning

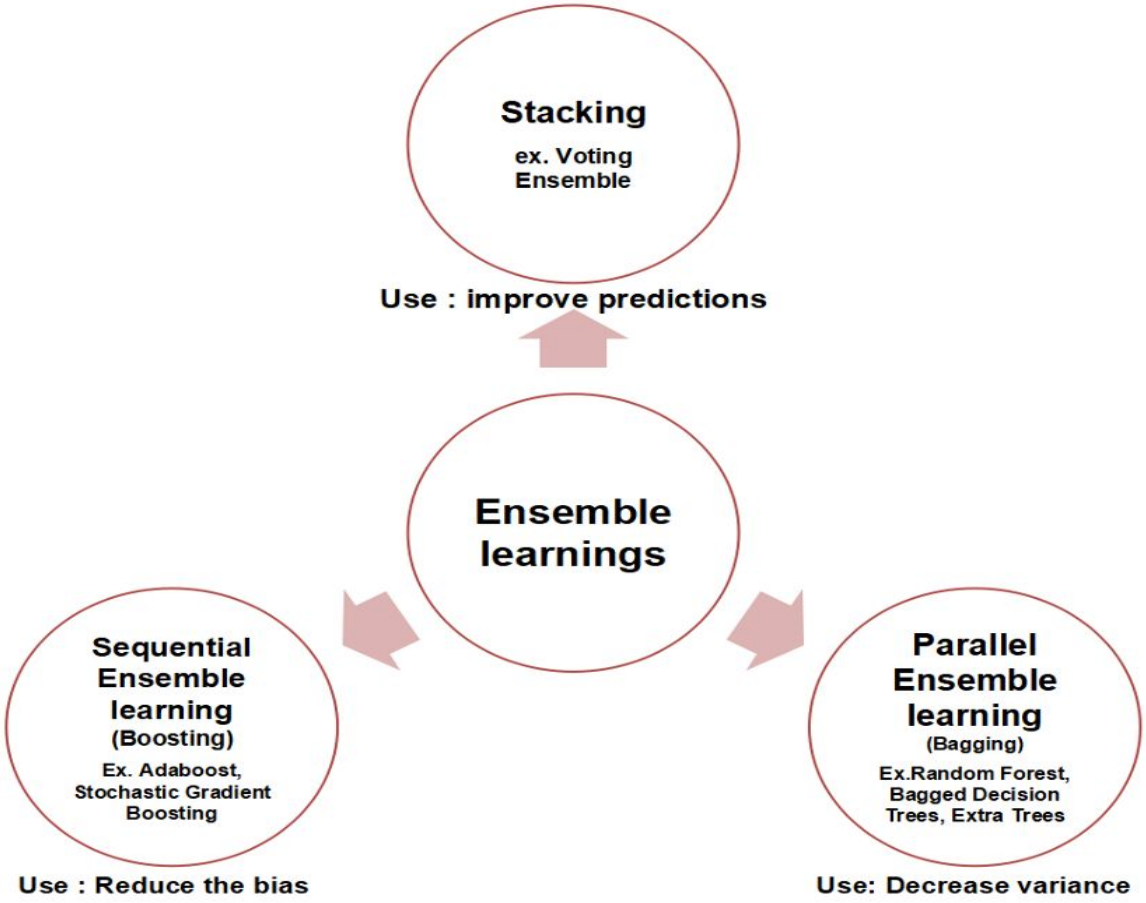
- Imagine you have the question of your life in front of you. Answering this question correctly will affect your life positively, otherwise negatively. But you're lucky: you can ask for advice from an expert of your choice, or you can ask an anonymous group, say 1,000 people, for advice. What advice would you get? The single expert opinion or the aggregated answer of an entire group of people? Or how about a group of experts?

- Ensemble Learning is therefore used in the hope that a group of algorithms will produce a better result on average than a single algorithm could

Types of Ensemble learning:



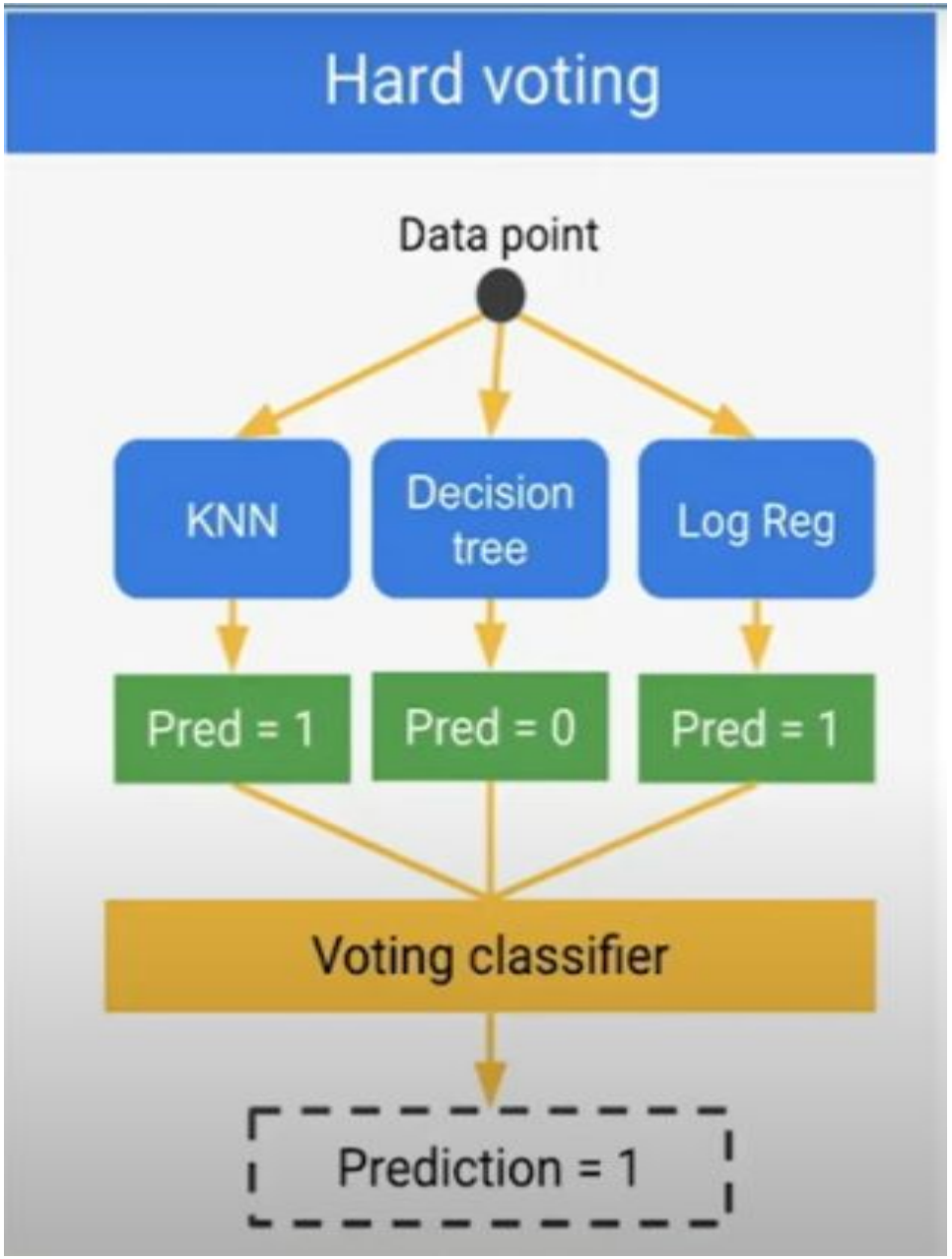
Ensembling Learning

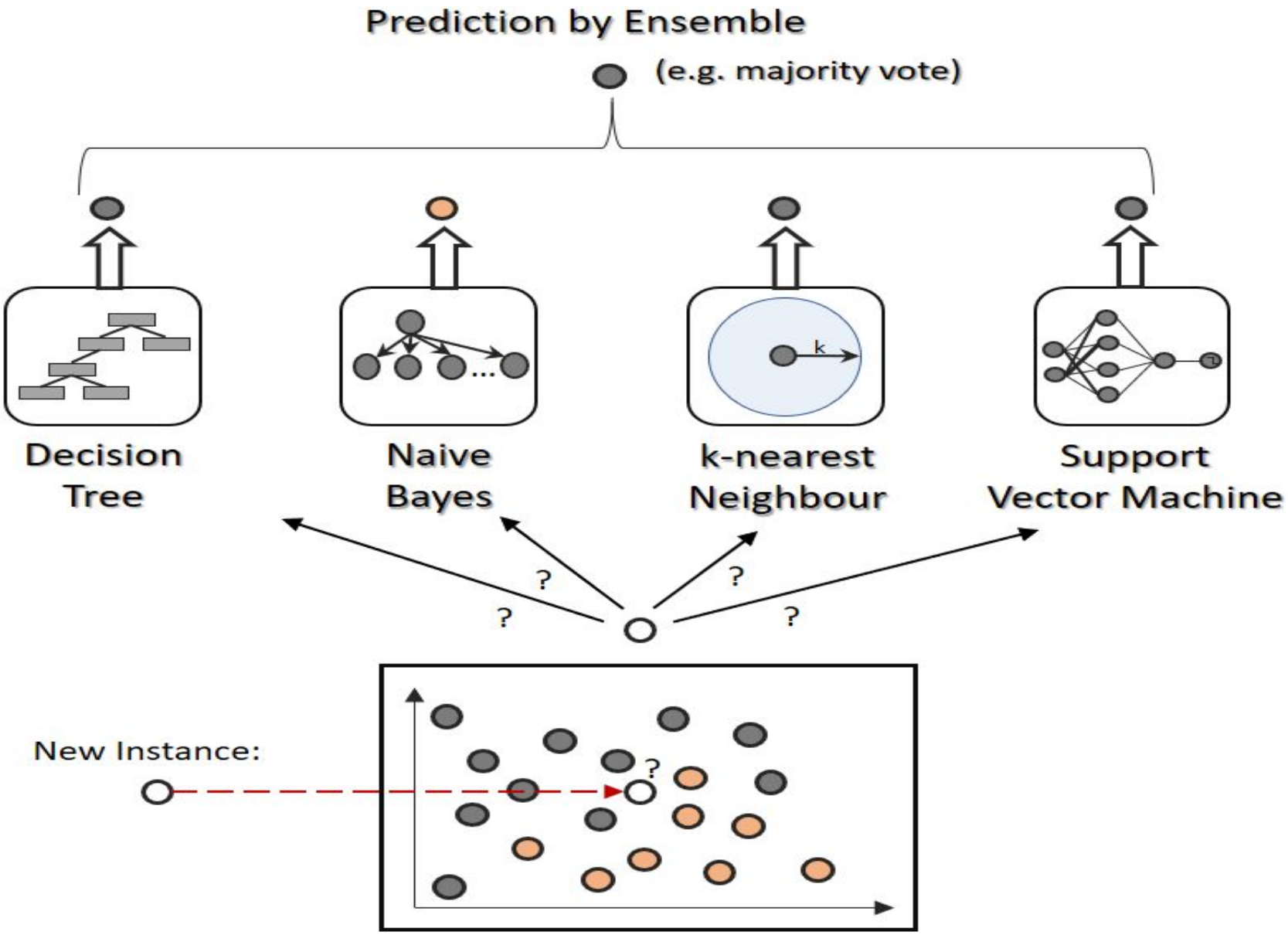


Voting Classifier

- A common form - and usually as the first example of an ensemble learner - is the principle of voting classifiers. The principle of voting classifiers is an extremely easy-to-understand idea of ensemble learning and is therefore probably one of the best-known forms of collective mean formation.
- A common question in data science is which classifier is better for certain purposes: decision trees, support vector machines, nearest neighbors or logistic regressions?
- Why not just use them all? In fact, this is often practiced. The goal of this form of ensemble learning is easy to see: the different weaknesses of all algorithms should - so hope - cancel each other out. All algorithms (this may also refer to

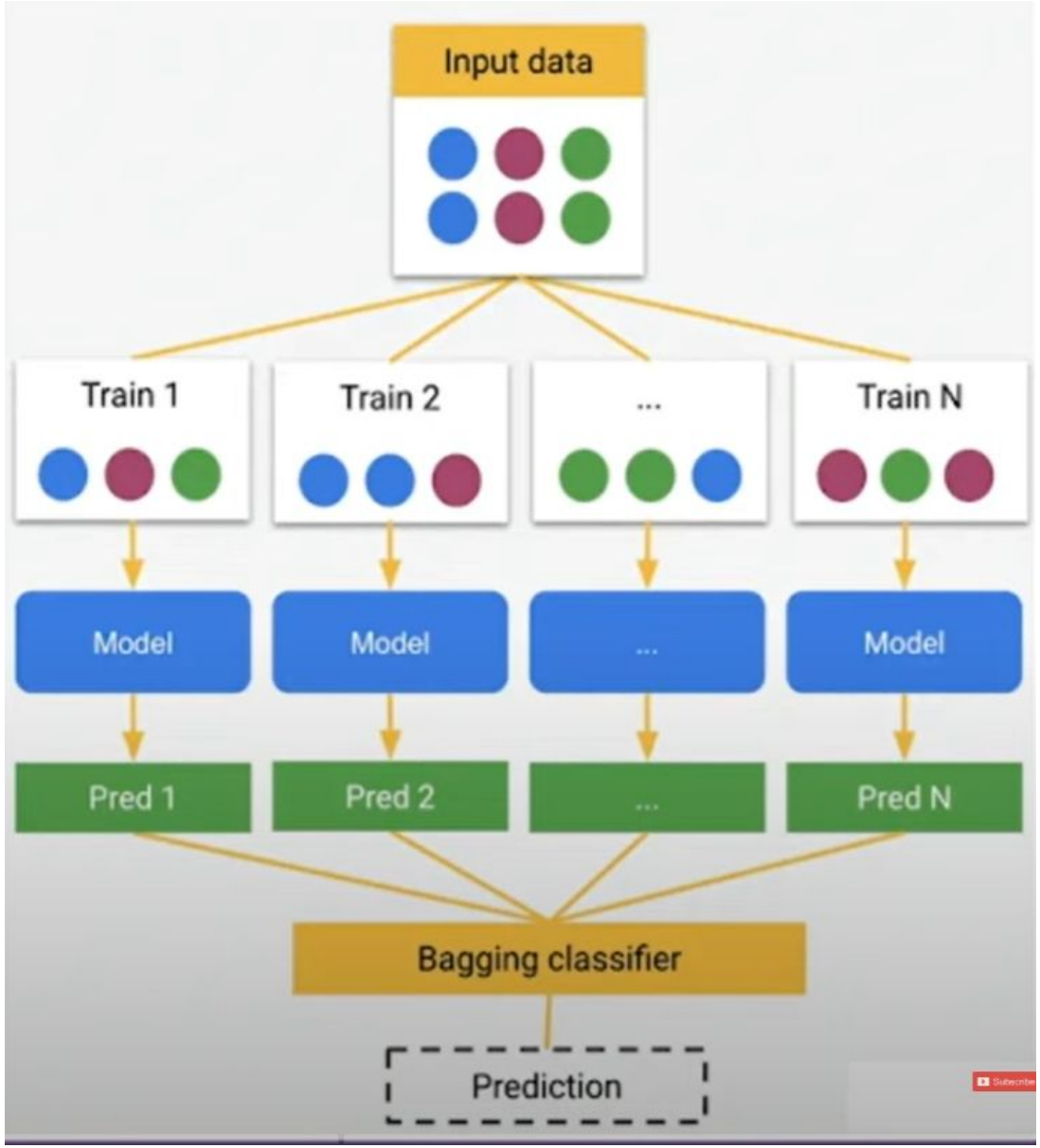
- Voting is an ensemble machine learning algorithm that involves making a prediction that is the average(regression) or the sum (classification) of multiple machine learning models
- First consider the multiple no. of the models, for each model is trained over the dataset

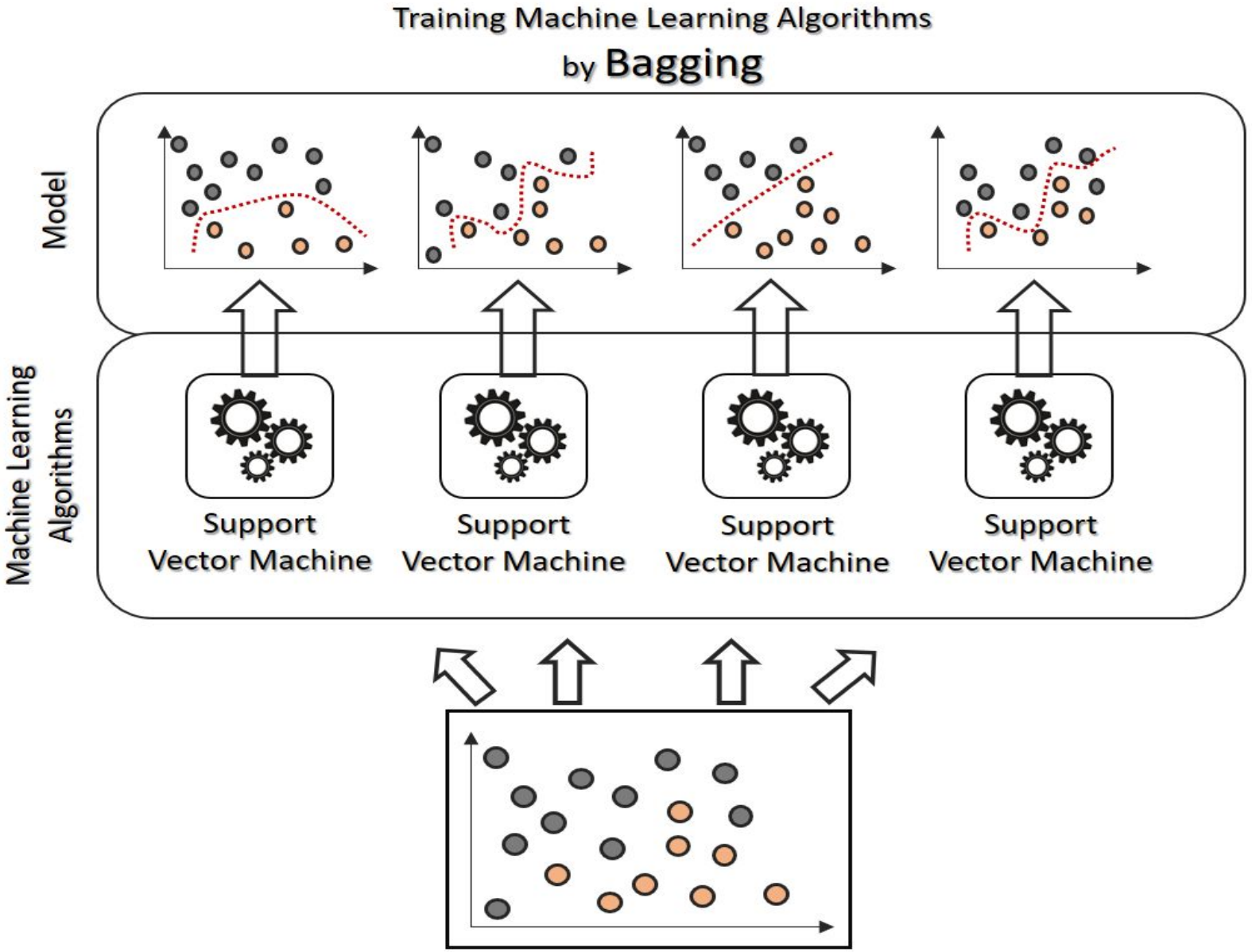



Prediction by Ensemble
(e.g. majority vote)
k
Decision Tree
Naive Bayes
k-nearest Neighbour
Support Vector Machine
?
?
?
?
New Instance:
?

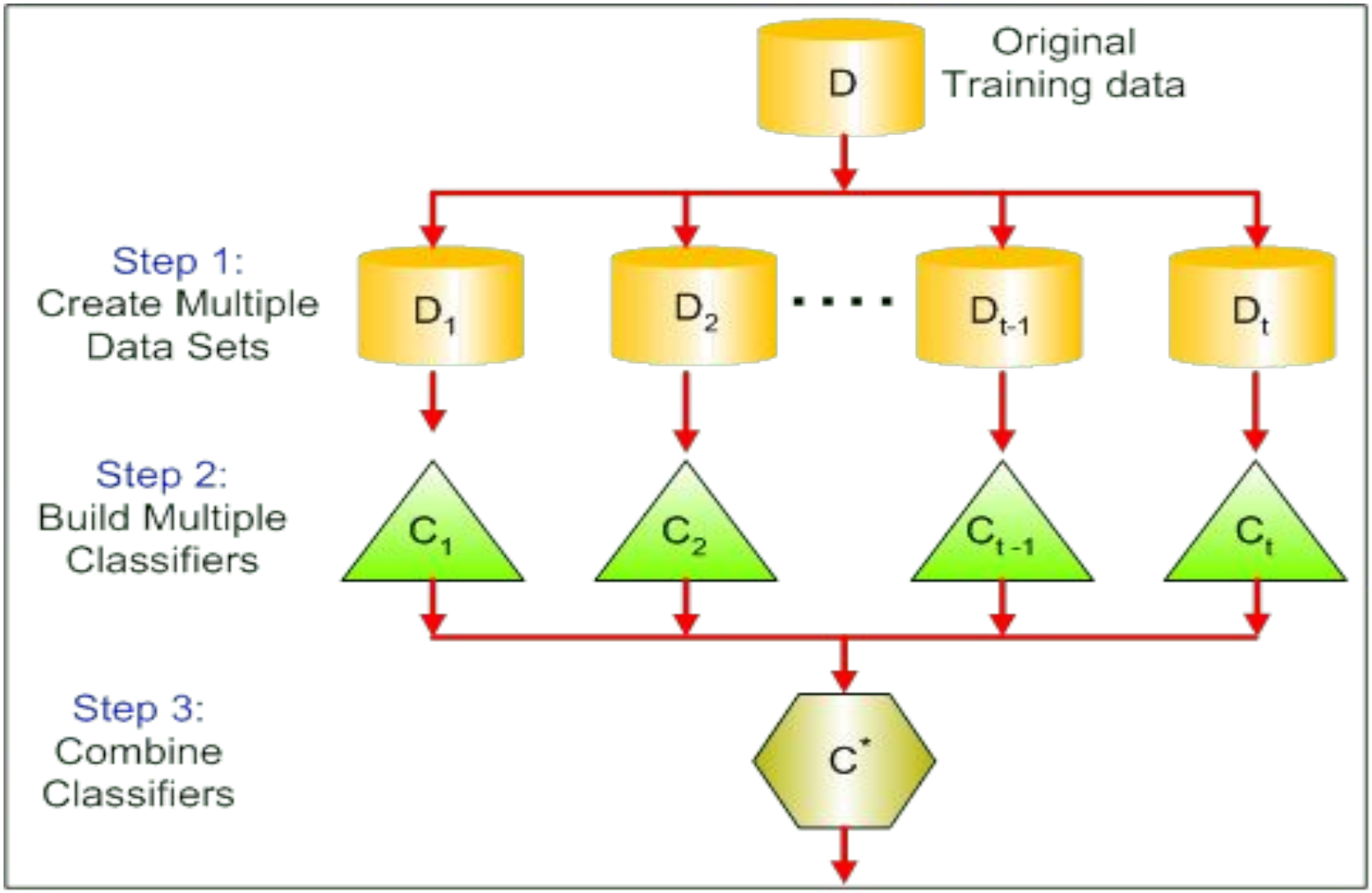
Bootstrap Aggregation (Bagging)

- Bootstrap aggregation also known as bagging
- Designed to improve stability and accuracy of machine learning algorithms like classification and regression
- It decreases variance and helps to avoid overfitting
- Usually applied to decision table tree methods



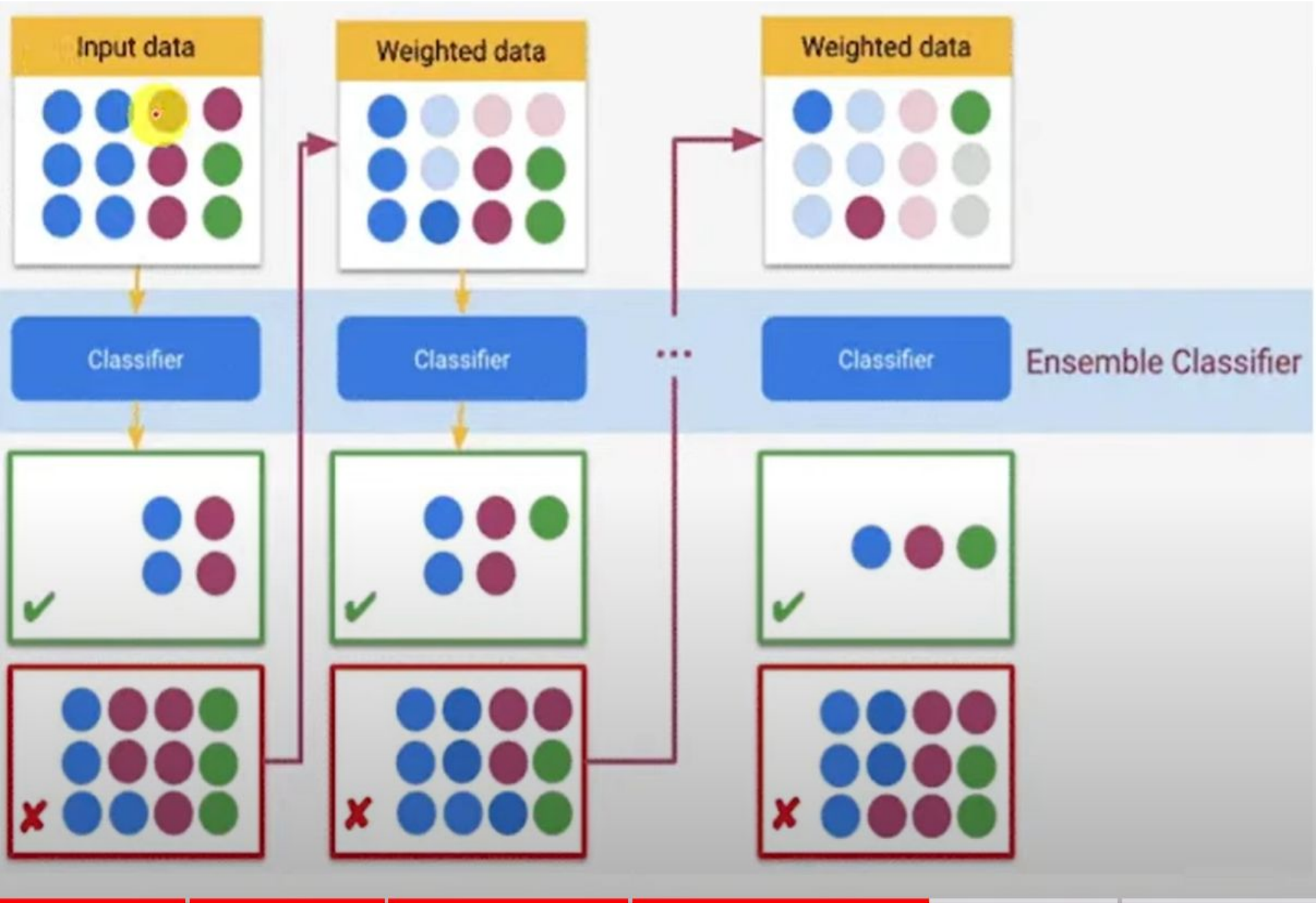

Training Machine Learning Algorithms
by Bagging
Model
Machine Learning Algorithms
Support Vector Machine
Support Vector Machine
Support Vector Machine
Support Vector Machine

Bagging (Bootstrap Aggregation)

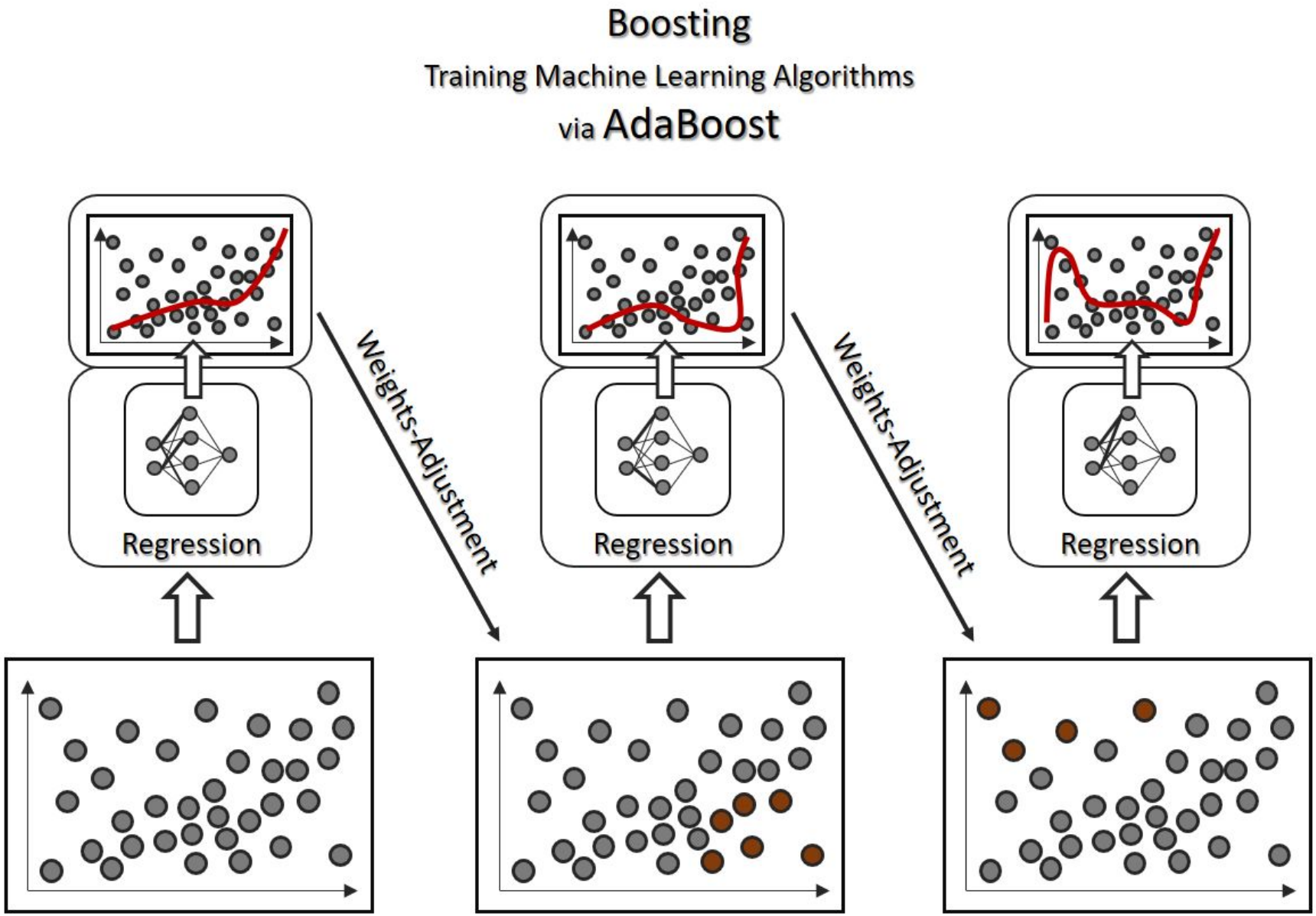


Boosting

- Boosting is an ensemble modeling technique that attempts to build a strong classifier from the no. of weak classifiers.
- It is done by building a model by using weak models in series
- Firstly, a model is built from the training data
- Then, the second model is built which tries to correct the errors present in the first model
- This procedure is continued and model are added until either the complete training data set is predicted correctly or the maximum number of models is added


Input data
Weighted data
Weighted data
Classifier
Classifier
Classifier
Ensemble Classifier

Boosting


Boosting
Training Machine Learning Algorithms
via AdaBoost
Regression
Weights-Adjustment
Regression
Weights-Adjustment
Regression

Blending (Stacked Generalization)

- Stacking, blending and stacked generalization are all the same thing with different names.
- In traditional ensemble learning, we have multiple classifiers trying to fit to a training set to approximate the target function
- Since each classifier will have its own output, we will need to find a combining mechanism to combine the results
- This can be through voting (majority wins). weighted voting (some classifier has more authority than the others) , averaging the results etc…

- In stacking, the combining mechanism is that the output of the classifiers (Level 0) will be used as training data for another classifier (Level 1) to approximate the same target function
- Basically, you let the level 1 classifier to figure out the combining mechanism

Basic Statistics

- Averages

  1)Mean

  2)Median

  3)Mode

- Variance and Covariance
- Bias
- Gaussian Function

1)Mean:

The "mean" is the average value of a dataset. It is calculated by adding up all the values in the dataset and dividing by the number of observations. The mean is a useful measure of central tendency because it is sensitive to outliers, meaning that extreme values can significantly affect the value of the mean.

In Python, we can calculate the mean using the **NumPy library,** which provides a function called **mean()**.

2) Median

The "median" is the middle value in a dataset. It is calculated by arranging the values in the dataset in order and finding the value that lies in the middle. If there are an even number of values in the dataset, the median is the average of the two middle values.

The median is a useful measure of central tendency because it is not affected by outliers, meaning that extreme values do not significantly affect the value of the median.

In Python, we can calculate the median using the **NumPy library,** which

3)Mode:

The "mode" is the most common value in a dataset. It is calculated by finding the value that occurs most frequently in the dataset. If there are multiple values that occur with the same frequency, the dataset is said to be bimodal, trimodal, or multimodal.

The mode is a useful measure of central tendency because it can identify the most common value in a dataset. However, it is not a good measure of central tendency for datasets with a wide range of values or datasets with no repeating values.

In Python, we can calculate the mode using the **SciPy library,** which provides a function called **mode()**.

```python
import numpy as np
import pandas as pd
# create a sample salary table
salary = pd.DataFrame({
    'employee_id': ['001', '002', '003', '004', '005', '006', '007',
    '008', '009', '010'],
    'salary': [50000, 65000, 45000, 45000, 70000, 60000, 55000, 45000,
    80000, 70000]
})

# calculate mean
mean_salary = np.mean(salary['salary'])
print('Mean salary:', mean_salary)

# calculate median
median_salary = np.median(salary['salary'])
print('Median salary:', median_salary)

# calculate mode
mode_salary = salary['salary'].mode()[0]
print('Mode salary:', mode_salary)
```

Edit & Run

Output

On executing this code, you will get the following output –

```
Mean salary: 59500.0
Median salary: 57500.0
```

Bias and Variance:

Machine learning is a branch of Artificial Intelligence, which allows machines to perform data analysis and make predictions. However, if the machine learning model is not accurate, it can make predictions errors, and these prediction errors are usually known as Bias and Variance.

- The main aim of ML/data science analysts is to reduce these errors in order to get more accurate results.

Bias:

In general, a machine learning model analyses the data, find patterns in it and make predictions. While training, the model learns these patterns in the dataset and applies them to test data for prediction. **While making predictions, a difference occurs between prediction values made by the model and actual values/expected values**, **and this difference is known as bias errors or Errors due to bias**

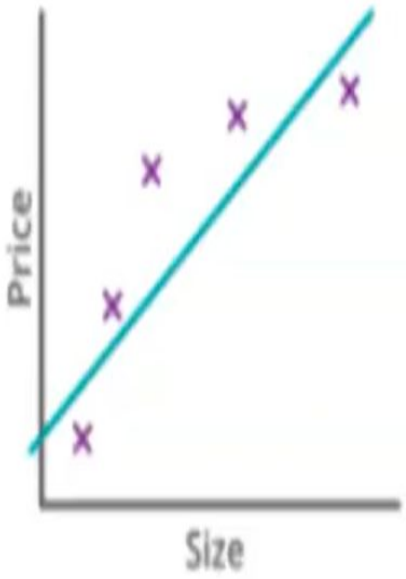
- Our model will not be trained well with the training data. there will be high training error when we train our model with the data

Variance :

- The variance would specify the amount of variation in the prediction **if the different training data** was used. In simple words, variance tells that how much a random variable is different from its expected value.

- If you train your data on training data and obtain a very low error, upon changing the data and then training the same previous model you experience high error. This is variance.

- Variance errors are either of low variance or high variance.
  - **Low variance** means there is a small variation in the prediction of the target function with changes in the training data set. At the same time.
  - **High variance** shows a large variation in the prediction of the target function with changes in the training dataset.



$$\theta_0 + \theta_1 x$$

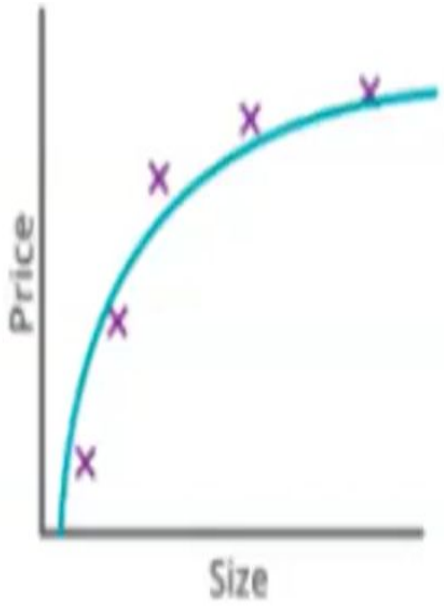
degree of polynomial = 1

High Training error

Bias - High

High Test error

Variance - High



$$\theta_0 + \theta_1 x + \theta_2 x^2$$

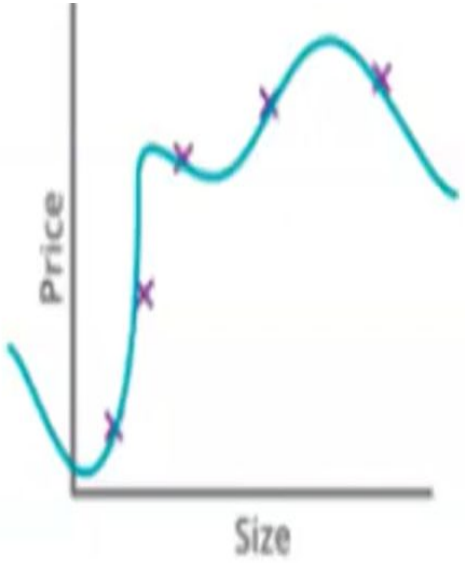
degree of polynomial = 2

Low Training error

Bias - Low

Low Test error

Variance - Low



$$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x$$

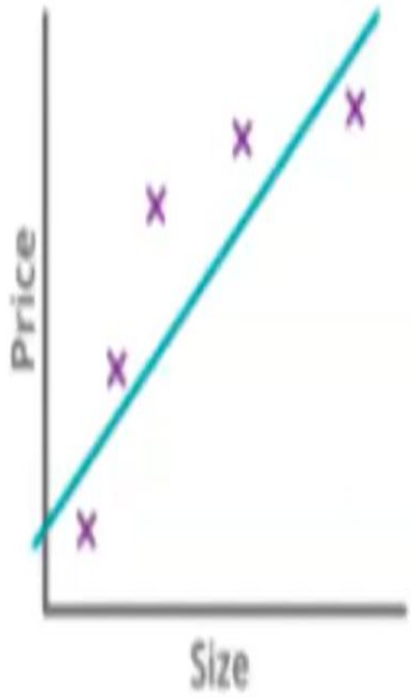
degree of polynomial = 4

Low Training error

Bias - Low

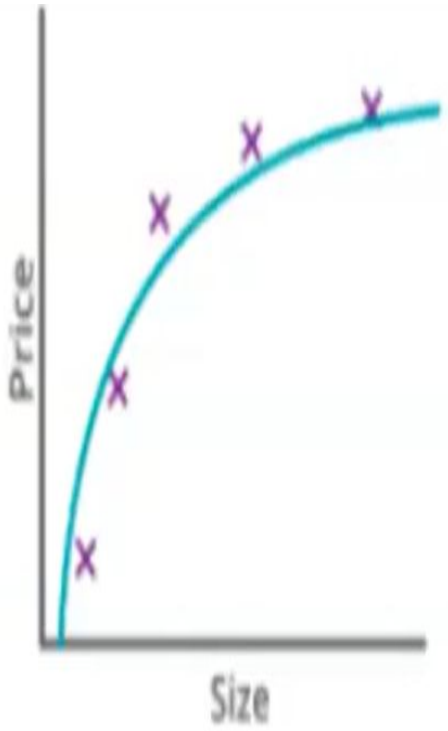
High Test error

Variance - High



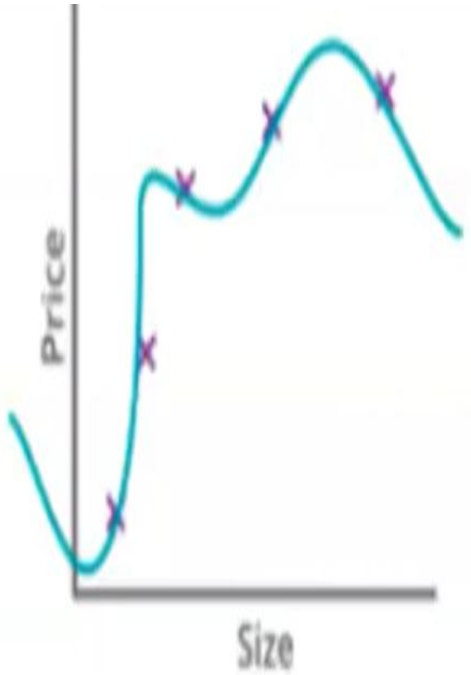
$$\theta_0 + \theta_1 x$$

Under Fitting



$$\theta_0 + \theta_1 x + \theta_2 x^2$$

Generalized Model



$$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x$$

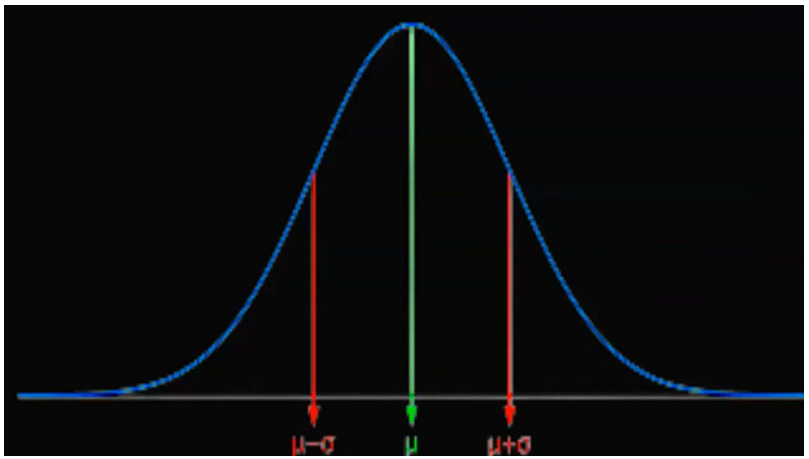
Over Fitting

Gaussian Function

- Also called as normal distribution/ Bell Curve
- If our machine learning models follows gaussian distribution, then it could predict outputs correctly with high accuracy.
- Probability density function

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$

- Bell curve



K-Nearest Neighbor(KNN) Algorithm

- ★ K-Nearest Neighbour is one of the simplest Machine Learning algorithms based on Supervised Learning technique.
- ★ K-NN algorithm assumes the similarity between the new case/data and available cases and put the new case into the category that is most similar to the available categories.
- ★ K-NN algorithm stores all the available data and classifies a new data point based on the similarity. This means when new data appears then it can be easily classified into a well suite category by using K- NN algorithm.
- ★ K-NN algorithm can be used for Regression as well as for Classification but mostly it is used for the Classification problems.

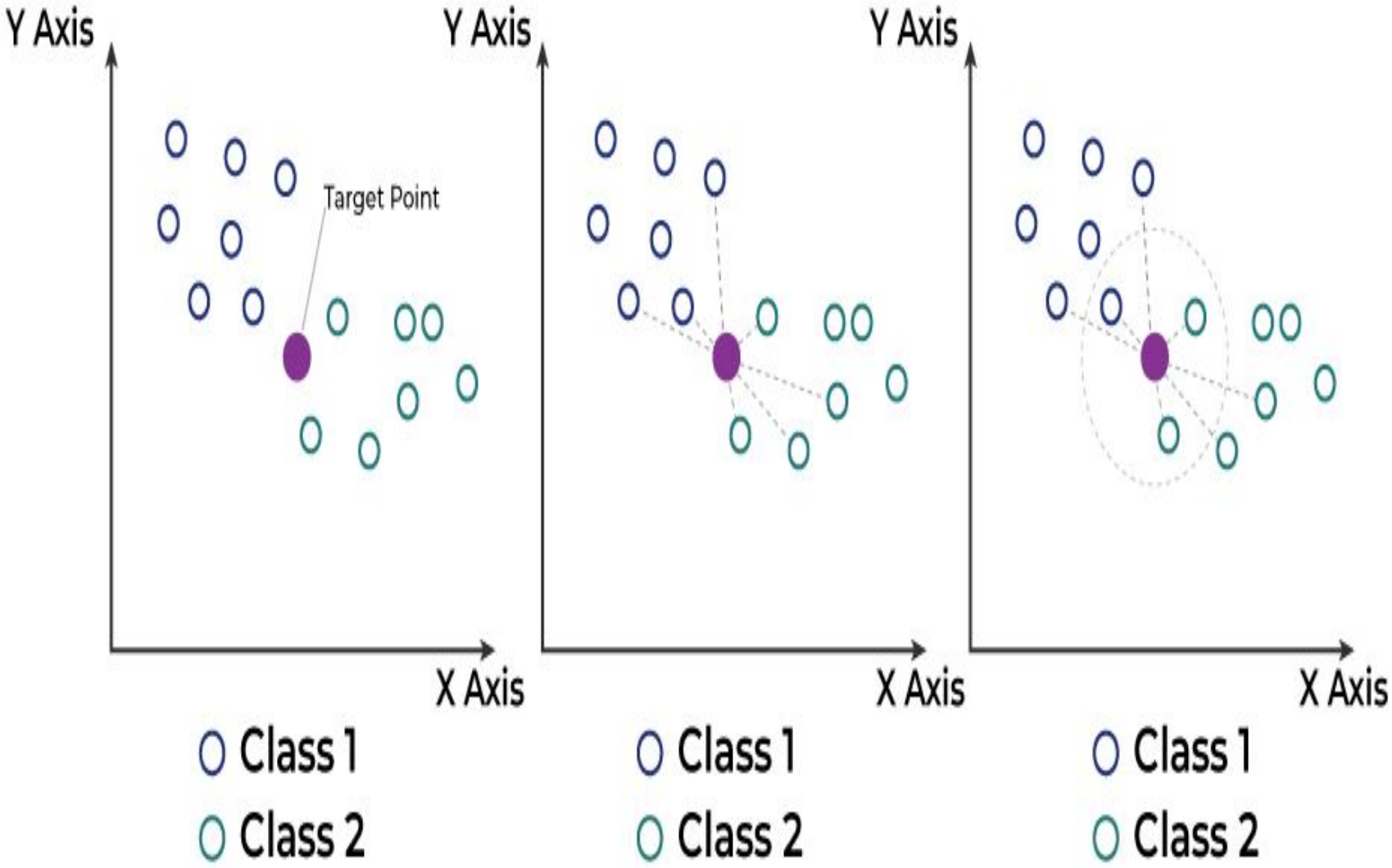
K-Nearest Neighbor(KNN) Algorithm

★ KNN algorithm at the training phase just stores the dataset and when it gets new data, then it classifies that data into a category that is much similar to the new data

★ This requires computing the distance to each datapoint in the training set, which is relatively expensive: if we are in normal Euclidean space, then we have to compute d subtractions and d squarings and this has to be done O(N2) times.

★ We can then identify the k nearest neighbours to the test point, and then set the class of the test point to be the most common one out of those for the nearest neighbours.

Steps:


Y Axis
Target Point
X Axis
Class 1
Class 2
Y Axis
X Axis
Class 1
Class 2
Y Axis
X Axis
Class 1
Class 2

Step 1: Selecting the optimal value of K

- K represents the number of nearest neighbors that needs to be considered while making prediction.

Step 2: Calculating distance

- To measure the similarity between target and training data points, Euclidean distance is used. Distance is calculated between each of the data points in the dataset and target point.

Step 3: Finding Nearest Neighbors

- The k data points with the smallest distances to the target point are the nearest neighbors.

Step 4: Voting for Classification or Taking Average for Regression

- In the classification problem, the class labels of are determined by performing majority voting. The class with the most occurrences among the neighbors becomes

- In the regression problem, the class label is calculated by taking average of the target values of K nearest neighbors. The calculated average value becomes the predicted output for the target data point.

The algorithm calculates the distance between x and each data point in X using a distance metric, such as Euclidean distance:

$$\text{distance}(x, X_i) = \sqrt{\sum_{j=1}^{d}(x_j - X_{i_j})^2]}$$

(or)

$$d_E = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2},$$

- Nearest neighbour methods are sensitive to noise, too large and the accuracy reduces as points that are too far away are considered. Some possible effects of changing the size of k on the decision boundary.

The k-nearest neighbours algorithm the bias-variance decomposition can be computed as:

$$E((\mathbf{y} - \hat{f}(\mathbf{x}))^2) = \sigma^2 + \left[f(\mathbf{x}) - \frac{1}{k} \sum_{i=0}^{k} f(\mathbf{x}_i) \right]^2 + \frac{\sigma^2}{k}.$$

Unsupervised Learning

- The aim of unsupervised learning is to find clusters of similar inputs in the data without being explicitly told that these data points belong to one class and those to a different class. Instead, the algorithm has to discover the similarities for itself.

K-Means Clustering Algorithm

- K-Means Clustering is an unsupervised learning algorithm that is used to solve the clustering problems
- K-Means Clustering is an Unsupervised Learning algorithm, which groups the unlabeled dataset into different clusters. Here K defines the number of pre-defined clusters that need to be created in the process, as if K=2, there will be two clusters, and for K=3, there will be three clusters, and so on.
- It allows us to cluster the data into different groups and a convenient way to discover the categories of groups in the unlabeled dataset on its own without the need for any training.
- It is a centroid-based algorithm, where each cluster is associated with a centroid. The main aim of this algorithm is to minimize the sum of distances between the data point and their corresponding clusters.
- The algorithm takes the unlabeled dataset as input, divides the dataset into k-number of clusters, and repeats the process until it does not find the best clusters. The value of k should be

The k-Means Algorithm

Find
K Means
Previous
Ne

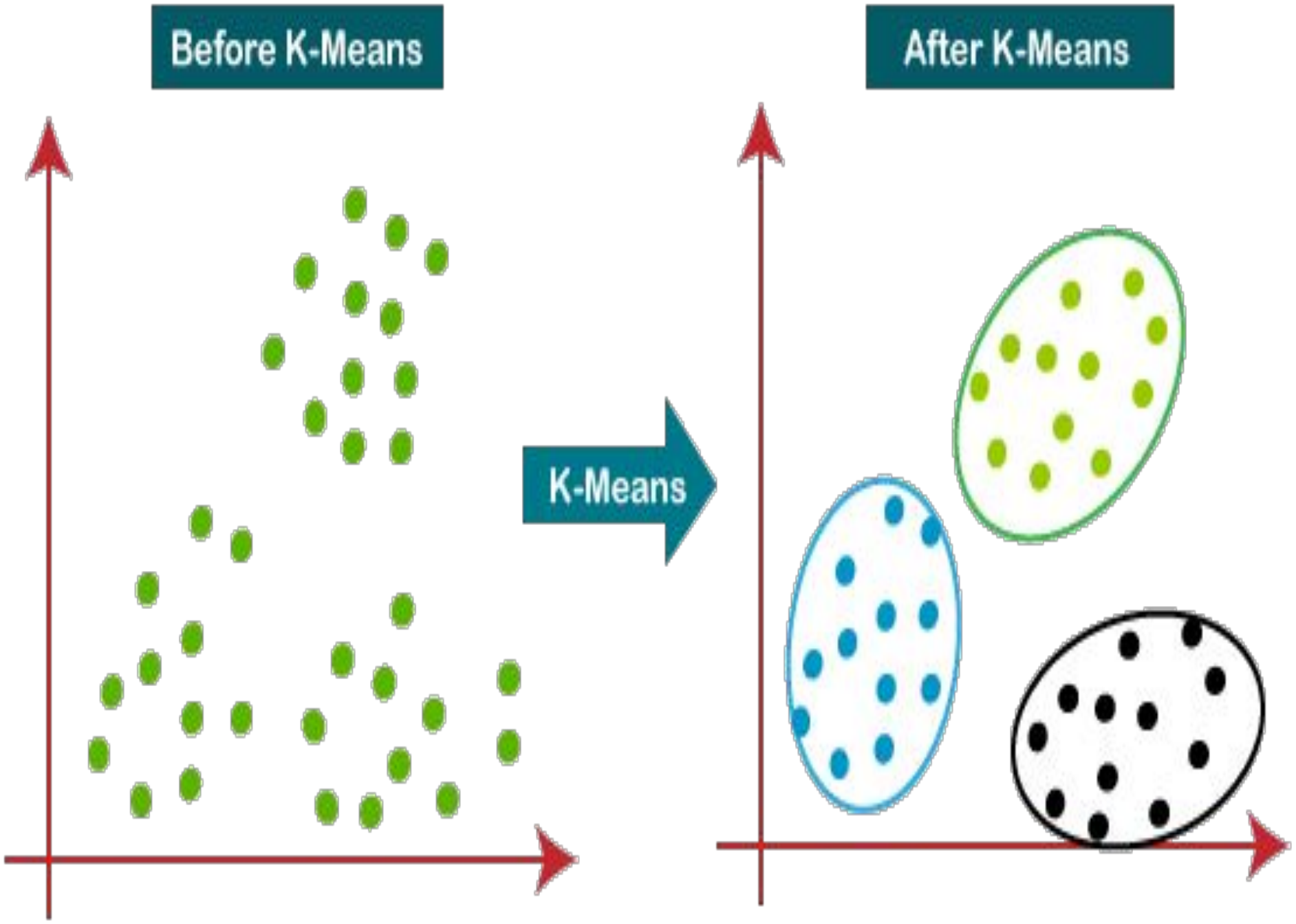
- **Initialisation**
  - choose a value for k
  - choose k random positions in the input space
  - assign the cluster centres $\boldsymbol{\mu}_j$ to those positions
- **Learning**
  - repeat
    - for each datapoint $\mathbf{x}_i$:
      - compute the distance to each cluster centre
      - assign the datapoint to the nearest cluster centre with distance

$$d_i = \min_j d(\mathbf{x}_i, \boldsymbol{\mu}_j). \tag{14.1}$$

    - for each cluster centre:
      - move the position of the centre to the mean of the points in that cluster (N_j is the number of points in cluster j):

$$\boldsymbol{\mu}_j = \frac{1}{N_j} \sum_{i=1}^{N_j} \mathbf{x}_i \tag{14.2}$$

  - until the cluster centres stop moving
- **Usage**
  - for each test point:
    - compute the distance to each cluster centre
    - assign the datapoint to the nearest cluster centre with distance


Before K-Means
K-Means
After K-Means

steps:

Step-1: Select the number K to decide the number of clusters.

Step-2: Select random K points or centroids. (It can be other from the input dataset).

Step-3: Assign each data point to their closest centroid, which will form the predefined K clusters.

Step-4: Calculate the variance and place a new centroid of each cluster.

Step-5: Repeat the third steps, which means reassign each datapoint to the new closest centroid of each cluster.

Step-6: If any reassignment occurs, then go to step-4 else go to FINISH.

Step-7: The model is ready.

Example:

- Suppose that the data mining task is to cluster points into three clusters,
- where the points are
- $A1(2, 10)$, $A2(2, 5)$, $A3(8, 4)$, $B1(5, 8)$, $B2(7, 5)$, $B3(6, 4)$, $C1(1, 2)$, $C2(4, 9)$.
- The distance function is Euclidean distance.
- Suppose initially we assign $A1$, $B1$, and $C1$ as the center of each cluster, respectively.

Initial Centroids:

A1: (2, 10)

B1: (5, 8)

C1: (1, 2)

Data Points			Distance to						Cluster	New Cluster
			2	10	5	8	1	2		
A1	2	10								
A2	2	5								
A3	8	4								
B1	5	8								
B2	7	5								
B3	6	4								
C1	1	2								
C2	4	9								

$$d(p_1, p_2) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Data Points			Distance to						Cluster	New Cluster
			2	10	5	8	1	2		
A1	2	10	0.00							
A2	2	5	5.00							
A3	8	4	8.49							
B1	5	8	3.61							
B2	7	5	7.07							
B3	6	4	7.21							
C1	1	2	8.06							
C2	4	9	2.24							

Data Points			Distance to						Cluster	New Cluster
			2	10	5	8	1	2		
A1	2	10	0.00		3.61		8.06			
A2	2	5	5.00		4.24		3.16			
A3	8	4	8.49		5.00		7.28			
B1	5	8	3.61		0.00		7.21			
B2	7	5	7.07		3.61		6.71			
B3	6	4	7.21		4.12		5.39			

Assign the data points to particular cluster, which is having smallest value

Data Points			Distance to						Cluster	New Cluster
			2	10	5	8	1	2		
A1	2	10	0.00		3.61		8.06		1	
A2	2	5	5.00		4.24		3.16		3	
A3	8	4	8.49		5.00		7.28		2	
B1	5	8	3.61		0.00		7.21		2	
B2	7	5	7.07		3.61		6.71		2	
B3	6	4	7.21		4.12		5.39		2	
C1	1	2	8.06		7.21		0.00		3	
C2	4	9	2.24		1.41		7.62		2	

Calculate the new centroid, update the initial centroid with new centroid.

Initial Centroids:
A1: (2, 10)
B1: (5, 8)
C1: (1, 2)

New Centroids:
A1: (2, 10)
B1: (6, 6)
C1: (1.5, 3.5)

Current Centroids:
A1: (2, 10)
B1: (6, 6)
C1: (1.5, 3.5)

Data Points				
			2	10
A1	2	10		
A2	2	5		
A3	8	4		
B1	5	8		
B2	7	5		
B3	6	4		
C1	1	2		
C2	4	9		

$d(p_1, p_2) =$

Again calculate the distance between the data points and assign to cluster

Current Centroids:
A1: (2, 10)
B1: (6, 6)
C1: (1.5, 3.5)

Data Points			Distance to						Cluster	New Cluster
			2	10	6	6	1.5	1.5		
A1	2	10	0.00		5.66		6.52		1	1
A2	2	5	5.00		4.12		1.58		3	3
A3	8	4	8.49		2.83		6.52		2	2
B1	5	8	3.61		2.24		5.70		2	2
B2	7	5	7.07		1.41		5.70		2	2
B3	6	4	7.21		2.00		4.53		2	2
C1	1	2	8.06		6.40		1.58		3	3
C2	4	9	2.24		3.61		6.04		2 →	1

Calculate new centroids

Current Centroids:
A1: (2, 10)
B1: (6, 6)
C1: (1.5, 3.5)

New Centroids:
A1: (3, 9.5)
B1: (6.5, 5.25)
C1: (1.5, 3.5)

Data Points			Distance to						Cluster	New Cluster
			2	10	6	6	1.5	1.5		
A1	2	10	0.00		5.66		6.52		1	1
A2	2	5	5.00		4.12		1.58		3	3
A3	8	4	8.49		2.83		6.52		2	2
B1	5	8	3.61		2.24		5.70		2	2
B2	7	5	7.07		1.41		5.70		2	2
B3	6	4	7.21		2.00		4.53		2	2
C1	1	2	8.06		6.40		1.58		3	3
C2	4	9	2.24		3.61		6.04		2	1

Current Centroids:
A1: (3, 9.5)
B1: (6.5, 5.25)
C1: (1.5, 3.5)

Data Points			Distance to						Cluster	New Cluster
			3	9.5	6.5	5.25	1.5	3.5		
A1	2	10	1.12		6.54		6.52		1	1
A2	2	5	4.61		4.51		1.58		3	3
A3	8	4	7.43		1.95		6.52		2	2
B1	5	8	2.50		3.13		5.70		2	1
B2	7	5	6.02		0.56		5.70		2	2
B3	6	4	6.26		1.35		4.53		2	2
C1	1	2	7.76		6.39		1.58		3	3
C2	4	9	1.12		4.51		6.04		1	1

★ In the above table, B1 point moved from cluster 2 to 1 so need to calculate the new centroid

New Centroids
A1: (3.67, 9)
B1: (7, 4.33)
C1: (1.5, 3.5)

Data Points			Distance to						Cluster	New Cluster
			3.67	9	7	4.33	1.5	3.5		
A1	2	10	1.94		7.56		6.52		1	1
A2	2	5	4.33		5.04		1.58		3	3
A3	8	4	6.62		1.05		6.52		2	2
B1	5	8	1.67		4.18		5.70		1	1
B2	7	5	5.21		0.67		5.70		2	2
B3	6	4	5.52		1.05		4.53		2	2
C1	1	2	7.49		6.44		1.58		3	3
C2	4	9	0.33		5.55		6.04		1	1